SolanaICO Audit Report

Version 1.0

Prepared by: Dionis Xhaferi

04/09/25

Table of Contents

Protocol Summary

This smart contract enables a simple token sale (ICO) using the Solana blockchain and the Anchor framework. It allows an admin to:

- Create a token account to hold sale tokens
- Deposit tokens into the program's account
- Let users buy tokens in exchange for SOL (at a fixed price)

Disclaimer

The DX team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Rust implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Scope

./finalcode/ #-- Contract | #-- SolanaICO.rs #-- pages | #-- _app.js | #-- index.js

Roles

- Admin: Initializes the ICO, deposits tokens, receives SOL payments.
- User (Buyer): Sends SOL to buy tokens from the ICO.

Executive Summary

The audit identified several issues in the ICO contract. Most notably, there is no validation of the admin account in the buy_tokens instruction, allowing attackers to redirect SOL payments to themselves. Additionally, the contract does not check whether enough tokens remain for sale before accepting a purchase, risking overselling. The Data account can also be reinitialized without restriction, potentially wiping existing ICO state. The lack of token balance checks, event emissions, and access control macros further weaken security, transparency, and maintainability.

Issues found

Severity	Number of issues found
High	2
Medium	2
Low	1
Info	1
Total	6

Findings

High / Critical

[H-1] Missing Admin Validation in `buy_tokens` (Broken Access Control + Fund Theft)

Description: The `buy_tokens` instruction does not validate that the `admin` account passed into the context matches the `admin` value stored in the program's `Data` account.

Impact: A malicious user can invoke `buy_tokens`, supply their own account as `admin`, and receive SOL from legitimate users. The program will still execute because token transfers use the program's signer, not the admin's. Anyone can call `buy_tokens` and pass in their own wallet as the `admin`, and the contract will happily send them the buyer's SOL.

Proof of Concept:

1. Bob builds or uses a front-end to call the buy_tokens instruction.
2. In the call, Bob passes:
    - Himself as the admin account.
    - A valid token amount (e.g. 100 tokens).
    - A valid PDA for the ico_ata_for_ico_program.
3. The program executes the following logic:

```
let ix = anchor_lang::solana_program::system_instruction::transfer(
    &ctx.accounts.user.key(),       // Buyer (e.g. Charlie)
    &ctx.accounts.admin.key(),      // Bob (not verified)
    sol_amount,
);
invoke(&ix, &[user_info, admin_info])?;
```

Key Problem: There's no validation to ensure that `ctx.accounts.admin` is Alice (the real admin) — it just blindly accepts the account and sends SOL to it.

4. Bob never had to hack the contract — he simply exploited the missing identity verification.

Recommended Mitigation: Add this validation to `buy_tokens`

```
if ctx.accounts.data.admin != *ctx.accounts.admin.key {
    return Err(error!(ErrorCode::InvalidAdmin));
}
```

[H-2] Unchecked Token Inventory During Purchase (Logic Flaw + Potential Oversell)

Description: The contract tracks how many tokens are sold `(tokens_sold)` and available `(total_tokens)`, but when a user buys tokens, there's no check to make sure tokens are still available before the purchase.

Impact: Users can attempt to buy more tokens than are available. The SOL will be accepted, but token transfer might fail, resulting in a loss or broken transaction.

Proof of Concept:

```
data.total_tokens = 100;
data.tokens_sold = 100;
buy_tokens(token_amount = 10); // Should fail but doesn't
```

Recommended Mitigation:

```
if data.tokens_sold + token_amount > data.total_tokens {
    return Err(error!(ErrorCode::Overflow)); // or custom error
}
```

Medium

[M-1] ICO Data Account Can Be Reinitialized (State Reset Vulnerability)

Description: The `create_ico_ata` instruction always initializes a new `Data` account using the admin's pubkey in the seed. It does not check if one already exists or has meaningful data.

Impact: Admin could accidentally or maliciously reinitialize the ICO, resetting `tokens_sold` and `total_tokens`, erasing historical state.

Proof of Concept:

```
invoke create_ico_ata(admin = same pubkey);
// Overwrites existing Data account state
```

Recommended Mitigation: Add this logic before overwriting the Data struct:

```
require!(data.tokens_sold == 0, CustomError::AlreadyInitialized);
```

[M-2] Lack of Token Balance Checks Before Transfer (Missing Validation)

Description: There are no checks to verify that the ICO ATA holds enough tokens before attempting a transfer. If insufficient, the transfer will fail midway after the SOL is already moved.

Impact: If the program account doesn't hold enough tokens (due to mismanagement or an earlier bug), the transfer fails — after SOL is already sent.

Proof of Concept:

- Buyer sends 100 SOL expecting 100 tokens.
- The program ATA only has 50 tokens due to an earlier withdrawal.
- The transfer of tokens fails.
- Buyer’s SOL is gone, and they didn’t get what they paid for.

```
ico_ata_for_ico_program.amount = 0;
buy_tokens(token_amount = 100); // Panics during token transfer
```

Recommended Mitigation: Add

```
require!(
    ico_ata_for_ico_program.amount >= raw_token_amount,
    CustomError::InsufficientICOFunds
```

```
);
```

Low

[L-1] No Event Emission on Key Actions (Observability Weakness)

Description: The contract does not emit Anchor events for major actions like ICO creation, token deposits, or token purchases. Indexers and explorers cannot track these changes.

Impact: Explorers like Solscan, indexers, and analytics tools can't see what's happening. You also lose visibility for audits, front-end monitoring, or dispute resolution.

Proof of Concept: N/A - architectural

Recommended Mitigation: Define events using `#[event]` and emit them:

```
#[event]
pub struct TokensPurchased {
    pub buyer: Pubkey,
    pub amount: u64,
    pub timestamp: i64,
}

emit!(TokensPurchased { ... });
```

Informational

[I-1] No Access Control Macros Used (Maintainability Concern)

Description: While you do enforce access control manually, you don't use Anchor's `#[access_control]` or helper guards. This makes the logic harder to follow and reason about.

Impact: Not a vulnerability but reduces clarity and increases cognitive load during audits or code reviews. It becomes harder to verify and extend access control logic.

Proof of Concept: N/A - style/design pattern

Recommended Mitigation:

```
#[access_control(is_admin(&ctx.accounts))]
pub fn deposit_ico_in_ata(...) { ... }

fn is_admin(accounts: &DepositIcoInATA) -> Result<()> {
    require!(accounts.data.admin == *accounts.admin.key, CustomError::InvalidAdmin);
    Ok(())
}
```